

Supervised learning: classification and model evaluation

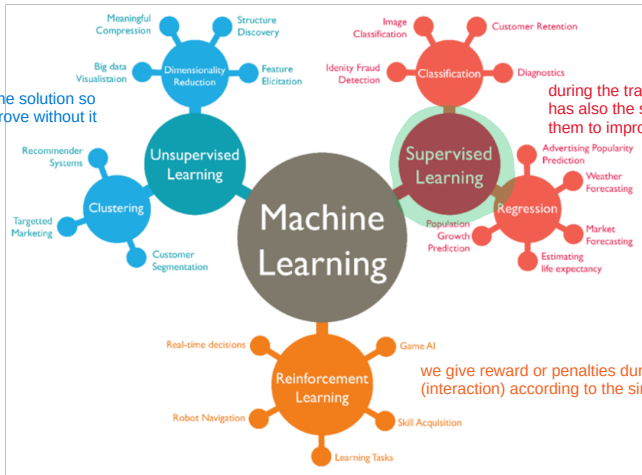
Stefano Diciotti¹

¹Dipartimento di Ingegneria dell'Energia Elettrica e dell'Informazione "Guglielmo Marconi" - DEI, Università di Bologna

Machine Learning Taxonomy

classification of all possible approaches to ML

we don't have the solution so we need to improve without it



during the training the algorithm has also the solution and we use them to improve the process

we give reward or penalties during the process (interaction) according to the single steps

Source: Ajit Jaokar, LinkedIn

labels = ground truth

The training data you feed to the algorithm include the desired output, called **labels**.

Typical task: predicting a future observation

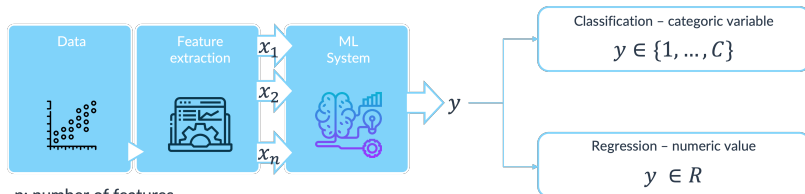
- **Classification** – the label is a categorical variable *e.g., benign vs. malignant lesion*
- **Regression** – the label is a numeric value *e.g., estimating cognitive decline based on MRI features*

Terminology Note

Subject vs. Instance

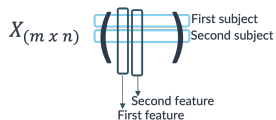
- In this course, we use the term **subject** to refer to an individual in a medical or clinical dataset — such as a patient, participant, or case.
- The term **instance** refers to a general input-output pair used in machine learning, regardless of the domain.
- For example:
 - A subject might be a patient with MRI, genetic, and clinical data.
 - An instance is a row in the dataset used for model training or evaluation.
- This distinction helps clarify when we refer to real-world medical data versus abstract ML concepts.

Supervised Learning



n: number of features
m: number of subjects

Design matrix:



Output array:



*“A computer program is said to learn from **experience E** with respect to some **task T** and some **performance measure P** , if its performance on T , as measured by P , improves with experience E .”*

— Tom Mitchell, 1997



Example in Autism Spectrum Disorder (ASD)

Regression Task

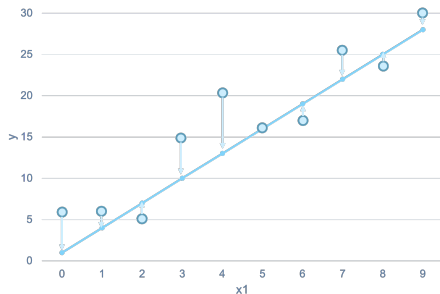
- **Task (T):** Predicting the ADOS score (Autism Diagnostic Observation Schedule)
- **Experience (E):** Genetic, neuroimaging, and clinical assessment data
- **Performance metric (P):** Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), etc.

Autism Diagnostic Observation Schedule (ADOS)

A standardized tool for assessing autism symptoms

- **ADOS** is a semi-structured, standardized assessment designed to evaluate communication, social interaction, play, and restricted or repetitive behaviors in individuals suspected of having Autism Spectrum Disorder (ASD).
- **Semi-structured** means the examiner follows a scripted set of activities, but adapts the interaction naturally based on the individual's responses — combining consistency with flexibility.
- The ADOS includes age- and language-level-appropriate activities, such as:
 - Free play or construction tasks (e.g., building a toy)
 - Asking about daily routines or emotions
 - Making a pretend phone call
 - Describing a picture or storytelling
- The assessment produces a **quantitative score** indicating the severity of autism-related symptoms, used in both clinical diagnosis and research (e.g., as a target variable in ML regression tasks).

Regression Performance Metrics



Predicting y using a single feature x_1 — regression line and prediction errors

Given y_i the true value, $\hat{y}_i$ the predicted value, and m the number of samples:

- **MAE** (Mean Absolute Error):

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$$

- **MSE** (Mean Squared Error):

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

- **RMSE** (Root Mean Squared Error):

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}$$

Performance Metrics in `scikit-learn`

Regression Task

Regression	
'explained_variance'	<code>metrics.explained_variance_score</code>
'max_error'	<code>metrics.max_error</code>
'neg_mean_absolute_error'	<code>metrics.mean_absolute_error</code>
'neg_mean_squared_error'	<code>metrics.mean_squared_error</code>
'neg_root_mean_squared_error'	<code>metrics.mean_squared_error</code>
'neg_mean_squared_log_error'	<code>metrics.mean_squared_log_error</code>
'neg_median_absolute_error'	<code>metrics.median_absolute_error</code>
'r2'	<code>metrics.r2_score</code>
'neg_mean_poisson_deviance'	<code>metrics.mean_poisson_deviance</code>
'neg_mean_gamma_deviance'	<code>metrics.mean_gamma_deviance</code>
'neg_mean_absolute_percentage_error'	<code>metrics.mean_absolute_percentage_error</code>
'd2_absolute_error_score'	<code>metrics.d2_absolute_error_score</code>
'd2_pinball_score'	<code>metrics.d2_pinball_score</code>
'd2_tweedie_score'	<code>metrics.d2_tweedie_score</code>

Overview of regression scoring options in `scikit-learn`'s model selection tools

Note: Scikit-learn uses negative error values (e.g., `neg_mean_squared_error`) to allow compatibility with score maximization logic.

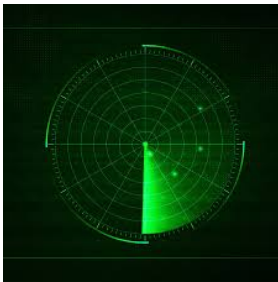
Example in Autism Spectrum Disorder (ASD)

Classification Task

- **Task (T):** Classification — ASD vs. Typical Development
- **Experience (E):** Genetic, neuroimaging, and clinical assessment data
- **Performance metric (P):** Sensitivity, specificity, accuracy, and AUC (area under the ROC curve)

Assessment of Classifier Performance: ROC Analysis (Receiver Operating Characteristic)

- ROC analysis is a standard method for evaluating and comparing the performance of classification models.
- Originally developed in signal detection theory to illustrate the trade-off between true positive rate (hit rate) and false positive rate (false alarm).



- Widely adopted in machine learning and medical diagnostics to analyze the behavior of decision systems.
- Addresses the limitations of relying solely on accuracy, especially in imbalanced datasets.

Dichotomous Classifier: Basic Concepts

- Let us consider a **dichotomous classifier**.
- Each instance is mapped to an element of the set $\{p, n\}$, representing the **positive** and **negative** classes, respectively.
- As a reference scenario, we consider a classification problem where:
 - An instance is a vector of features describing a subject.
 - The **positive class** (p) indicates the **presence of a disease**.
 - The **negative class** (n) indicates the **absence of the disease**.
- Some classifiers output a **continuous score** (e.g., probability estimates), which requires applying a **threshold** to decide class membership.
- Other classifiers produce a **discrete class label** directly.
- Let us denote the predicted classes with the set $\{Y, N\}$:
 - Y means the classifier predicts the **subject has a disease**.
 - N means the classifier predicts the **subject is healthy**.

Confusion Matrix (or Contingency or Stimulus-Response Matrix)

- Given a **dichotomous classifier** and a single instance, there are four possible outcomes:
 - The instance is positive and classified as positive: **True Positive (TP)**
 - The instance is positive and classified as negative: **False Negative (FN)**
 - The instance is negative and classified as negative: **True Negative (TN)**
 - The instance is negative and classified as positive: **False Positive (FP)**
- Given a classifier and a set of instances (test set), a **confusion matrix** (2×2) can be constructed:

Actual \ Predicted	Positive (Y)	Negative (N)
Positive (p)	True Positive (TP)	False Negative (FN)
Negative (n)	False Positive (FP)	True Negative (TN)

Confusion Matrix (Probabilistic View)

- Given a **dichotomous classifier** and a set of instances, we can define a 2x2 confusion matrix in terms of **ratios/probabilities**.
- Each cell represents a **conditional probability**, e.g., $P(Y | p)$ = probability the classifier predicts positive given the instance is actually positive.

Actual \ Predicted	Positive (Y)	Negative (N)
Positive (p)	$P(Y p)$ TP rate	$P(N p)$ FN rate
Negative (n)	$P(Y n)$ FP rate	$P(N n)$ TN rate

- These probabilities are estimated from the test set as relative frequencies.
Class rates:
 - TP rate** = Positives correctly classified / Total positives
 - FP rate** = Negatives incorrectly classified / Total negatives
 - TN rate** = Negatives correctly classified / Total negatives
 - FN rate** = Positives incorrectly classified / Total positives
- They reflect the behavior of the classifier under known ground-truth labels.

Confusion Matrix: Worked Example

Inst no.	Class		Score
	True	Hyp	
1	p	Y	0.99999
2	p	Y	0.99999
3	p	Y	0.99993
4	p	Y	0.99986
5	p	Y	0.99964
6	p	Y	0.99955
7	n	Y	0.68139
8	n	Y	0.50961
9	n	N	0.48880
10	n	N	0.44951

Example:

6 positive instances, 4 negative instances
Threshold = 0.5 (i.e., if score > 0.5 then
predicted = Y)

- True Positives rate: $6/6 = 1.0$ (100%)
- True Negatives rate: $2/4 = 0.5$ (50%)
- False Positives rate: $2/4 = 0.5$ (50%)
- False Negatives rate: $0/6 = 0.0$ (0%)

Probabilistic Interpretation of the Confusion Matrix

- If the classifier is **perfect**, we obtain only correct predictions, i.e., the following conditional probabilities hold:
 - $P(Y | p) = 1, \quad P(N | p) = 0$
 - $P(Y | n) = 0, \quad P(N | n) = 1$
- If the classifier is **imperfect**, these probabilities (estimated as relative frequencies) take values strictly between 0 and 1.
- The probabilities are related by the following identities:
 - $P(Y | p) + P(N | p) = 1$
 - $P(Y | n) + P(N | n) = 1$
- **Conclusion:** Only two elements of the confusion matrix are independent — the other two can be derived.

Metrics Derived from the Confusion Matrix

- **Sensitivity (or Recall or True Positive Rate):** proportion of actual positives correctly identified

$$\text{TPR} = P(Y | p) = \frac{TP}{TP + FN}$$

- **Specificity (or True Negative Rate):** proportion of actual negatives correctly identified

$$\text{TNR} = P(N | n) = \frac{TN}{TN + FP}$$

- **Precision (or Positive Predictive Value):** proportion of predicted positives that are actually positive

$$\text{Precision} = \text{PPV} = P(p | Y) = \frac{TP}{TP + FP}$$

- A good classifier should ideally have high values for sensitivity, specificity, and precision.

Why F1-Score is Often Better than Accuracy

- **Accuracy** measures the overall proportion of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- In **imbalanced datasets**, accuracy can be misleading:
 - E.g., if 95% of instances are negative, a classifier that always predicts negative gets 95% accuracy — but fails to detect any positives.
- **F1-score** balances **Precision** and **Recall**:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why Use the Harmonic Mean in F1-Score?

- The **F1-score** is defined as the harmonic mean of Precision and Recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Example:** Suppose a classifier has:

$$\text{Precision} = 1.0, \quad \text{Recall} = 0.1$$

- **Arithmetic mean:**

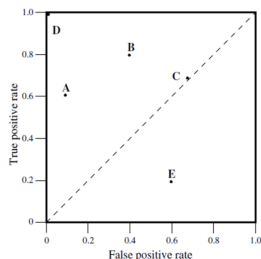
$$\frac{1.0 + 0.1}{2} = 0.55 \quad (\text{seems acceptable})$$

- **Harmonic mean (F1-score):**

$$F1 = 2 \cdot \frac{1.0 \cdot 0.1}{1.0 + 0.1} = 0.18 \quad (\text{reflects imbalance})$$

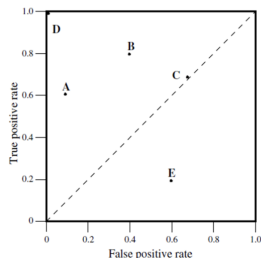
- **Conclusion:** The harmonic mean **penalizes large differences** between Precision and Recall, highlighting poor performance when either metric is low.

ROC Plane and the Confusion Matrix



- The confusion matrix has only **two degrees of freedom** (two independent values).
- All the information it contains can be mapped to a point *A* in a **2D graph**:
 - **x-axis**: False Positive Rate (FPR)
 - **y-axis**: True Positive Rate (TPR)
- **ROC Plane**: plots TPR vs. FPR.
- A **dichotomous classifier** is represented by a **point** in this ROC space.
- The point (0, 0): classifier always predicts "N".
- The point (1, 1): classifier always predicts "Y".
- The point (0, 1): **perfect classification**.

Interpreting Points in the ROC Plane



- Informally, one point in the ROC plane is considered **better** than another if it lies **higher and to the left**.
- Classifiers **down and to the left** are more **conservative**:
 - They make positive predictions only when there is strong evidence.
 - Result: few false positives (FPs), but also low true positives (TPs).
- Classifiers **up and to the right** are more **liberal**:
 - They tend to predict the positive class even with weak evidence.
 - Result: many true positives, but also many false positives.
- In many real-world problems with a majority of negative instances, performance in the **left region** of the ROC space is often more important.

Random Classifier in the ROC Plane

- A **random classifier** makes predictions without using any informative pattern from the data.
- For example, if it predicts the **positive class 50% of the time**, it will correctly classify:
 - 50% of the actual positives $\rightarrow$ TPR = 0.5
 - 50% of the actual negatives as false positives $\rightarrow$ FPR = 0.5 $\Rightarrow$ Point (0.5, 0.5) in the ROC plane
- If it predicts positive **90% of the time**, it will produce:

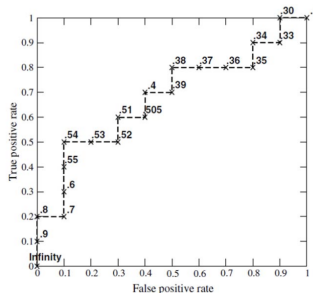
$$\text{TPR} = 0.9, \quad \text{FPR} = 0.9 \Rightarrow \text{Point (0.9, 0.9)}$$

- Therefore, the random classifier generates points along the **diagonal** of the ROC plane (from (0,0) to (1,1)), depending on how often it predicts "positive".
- Any classifier that lies **below this diagonal** performs worse than random — this is rare and often indicates systematic errors or reversed predictions.
- A random classifier has **no discriminative power** — it cannot distinguish between positive and negative instances beyond chance level.

- Many classifiers (e.g., decision trees) produce a **single class label** (Y or N) for each instance.
 - When such a **dichotomous classifier** is applied to a test set, it yields a **confusion matrix**, which corresponds to a **single point** in the ROC plane.
- Other classifiers (e.g., neural networks, logistic regression) output a **score or probability** for each instance — a value indicating the degree of confidence in the positive class.
- This score can be turned into a binary decision using a **threshold**:
 - If score $>$ threshold $\rightarrow$ predict "Y"
 - Else $\rightarrow$ predict "N"
- Each threshold value produces a different point in the ROC plane.
- By **varying the threshold** from $-\infty$ to $+\infty$, we can generate a series of points and trace a **curve in the ROC plane** — the **ROC curve**.

Example: Building the ROC Curve from Scores

Inst#	Class	Score	Inst#	Class	Score
1	p	.9	11	p	.4
2	p	.8	12	n	.39
3	n	.7	13	p	.38
4	p	.6	14	n	.37
5	p	.55	15	n	.36
6	p	.54	16	n	.35
7	n	.53	17	p	.34
8	n	.52	18	n	.33
9	p	.51	19	p	.30
10	n	.505	20	n	.1



- Any ROC curve generated by a finite number of instances is a **step curve**.
- As the number of instances tends to $+\infty$, the ROC curve approaches a **smooth, continuous curve**.
- A threshold of $+\infty$ corresponds to predicting all instances as negative $\rightarrow$ point (0,0) in the ROC plane.
- As the threshold decreases, more instances are predicted positive:
 - TPR increases, but so does FPR $\rightarrow$ curve moves **up and to the right**.
 - This transition reflects the move from a **conservative** to a **liberal** classification behavior.

Generating an ROC Curve from a Dichotomous Classifier

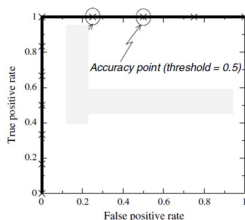
- If we want to generate an **ROC curve** from a **dichotomous classifier** (i.e., one that only outputs "Y" or "N"), we need to obtain a **score** rather than just a class label.
- A single class label yields a **single point** in the ROC plane — not a full curve.
- Fortunately, many dichotomous classifiers can be converted into **scoring classifiers** by "**looking inside**":
 - For example, decision trees can expose the **proportion of positive samples in a leaf**.
 - k-NN classifiers can use the **fraction of neighbors** in the positive class.
 - Rule-based systems may have internal **confidence values**.
- These internal values can be used as scores to vary the threshold and generate the full ROC curve.

Why Generate Scores from Classifiers with Predicted Labels?

- Some classifiers (e.g., logistic regression, neural networks) estimate the **probability of class membership** and then apply a decision rule:

If $p > 0.5$ then predict Y

- However, the **probability estimates may not be perfectly calibrated**.
- Using a fixed threshold (e.g., 0.5) might not yield optimal performance.
- It is often beneficial to **explore alternative thresholds** and analyze the trade-offs between true positives and false positives.
- The ROC curve allows us to evaluate classifier behavior across the full range of thresholds.



Inst no.	Class		Score
	True	Hyp	
1	p	Y	0.99999
2	p	Y	0.99999
3	p	Y	0.99993
4	p	Y	0.99986
5	p	Y	0.99964
6	p	Y	0.99955
7	n	Y	0.68139
8	n	Y	0.50961
9	n	N	0.48880
10	n	N	0.44951

- **Calibration** means that the predicted probabilities match the observed frequencies.
- Example: if a classifier assigns a probability of 0.7 to 100 instances, then about 70 of them should actually belong to the positive class.
- Calibration is important when:
 - Decisions are made based on predicted probabilities, not just class labels
 - We need to estimate risk or uncertainty reliably

- ROC curves have a very desirable property: they are **insensitive to changes in class distribution**.
 - If the proportion of positive and negative instances in the test set changes, the ROC curve remains the same.
- Class distribution corresponds to the relationship between the **two rows** of the confusion matrix:
 - First row: actual positives
 - Second row: actual negatives
- Metrics that use values from **both rows** — such as **accuracy** — are affected by class imbalance.
- As the class distribution changes, those metrics will also change, **even if the classifier's behavior stays the same**.
- ROC curves are based on:
 - **TPR (from the first row)** and **FPR (from the second row)**.→ **They are independent of the class distribution.**

ROC Curves: Example of Class Imbalance Effect

Example: a classifier with the following fixed performance:

- Sensitivity (True Positive Rate): **10%**
- Specificity (True Negative Rate): **90%**

Now observe what happens to the **accuracy** as the class distribution varies:

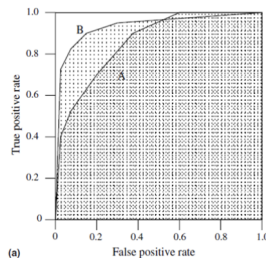
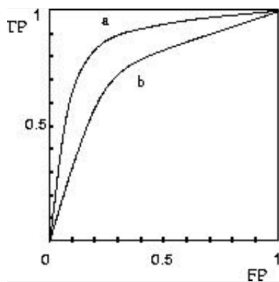
- **100 positive, 100 negative** → Accuracy = $(10 + 90)/(100 + 100) = 50\%$
- **100 positive, 1000 negative** → Accuracy = $(10 + 900)/(100 + 1000) = 83\%$
- **100 positive, 10,000 negative** → Accuracy = $(10 + 9000)/(100 + 10,000) = 89\%$

Conclusion: Accuracy increases with more negative cases, **even though the classifier's performance (TPR and TNR) stays the same**. This is why ROC curves are preferred: they are **invariant to class distribution**.

Area Under the ROC Curve (AUC)

- The ROC curve is a 2D plot that summarizes the performance of a classifier across all thresholds.
- To compare classifiers with a single number, we compute the **Area Under the ROC Curve (AUC)**.
- AUC ranges from 0 to 1:
 - AUC = 0.5 for a **random classifier**
 - AUC $>$ 0.5 for any classifier with predictive value
 - AUC = 1.0 for a **perfect classifier**
- **Statistical interpretation:** AUC is the probability that the classifier assigns a higher score to a randomly chosen positive instance than to a randomly chosen negative one.
- AUC is threshold-independent and robust to class imbalance.

ROC Curve Comparison



- The **ideal ROC curve** connects the points (0,0), (0,1), and (1,1), enclosing the maximum possible area (AUC = 1).
- If two ROC curves do **not intersect**, the one that lies consistently above the other represents the **better classifier** (i.e., higher AUC).
- If the ROC curves **intersect**, the comparison depends on the application's priorities:
 - Focus may be on regions with high specificity (low FPR) or high sensitivity (high TPR).
 - In such cases, the **partial AUC** within the relevant range is more informative.

Performance Metrics in `scikit-learn`

Classification Task

Scoring Classification	Function	Comment
'accuracy'	<code>metrics.accuracy_score</code>	
'balanced_accuracy'	<code>metrics.balanced_accuracy_score</code>	
'top_k_accuracy'	<code>metrics.top_k_accuracy_score</code>	
'average_precision'	<code>metrics.average_precision_score</code>	
'neg_brier_score'	<code>metrics.brier_score_loss</code>	
'f1'	<code>metrics.f1_score</code>	for binary targets
'f1_micro'	<code>metrics.f1_score</code>	micro-averaged
'f1_macro'	<code>metrics.f1_score</code>	macro-averaged
'f1_weighted'	<code>metrics.f1_score</code>	weighted average
'f1_samples'	<code>metrics.f1_score</code>	by multilabel sample
'neg_log_loss'	<code>metrics.log_loss</code>	requires <code>predict_proba</code> support
'precision' etc.	<code>metrics.precision_score</code>	suffixes apply as with 'f1'
'recall' etc.	<code>metrics.recall_score</code>	suffixes apply as with 'f1'
'jaccard' etc.	<code>metrics.jaccard_score</code>	suffixes apply as with 'f1'
'roc_auc'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovr'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovo'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovr_weighted'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovo_weighted'	<code>metrics.roc_auc_score</code>	

Overview of classification scoring options in `scikit-learn`'s model selection tools

- T. Fawcett, "An introduction to ROC analysis", Pattern Recognition Letters, Volume 27, Issue 8, June 2006, Pages 861-874